

Task 23: Vibe Coding and Prompt Engineering

AIM

To apply Vibe Coding and Prompt Engineering techniques using a Generative AI tool to design effective prompts for generating code snippets, cloud configuration templates, and application logic, and evaluate the output quality.

ALGORITHM

- Step 1: Understand the principles of Vibe Coding and Prompt Engineering.
- Step 2: Design a prompt to generate a REST API code snippet.
- Step 3: Design a prompt to generate a cloud deployment configuration (e.g., Dockerfile).
- Step 4: Design a prompt to generate application business logic.
- Step 5: Evaluate each generated output for correctness, efficiency, and usability.
- Step 6: Refine prompts iteratively to improve output quality.
- Step 7: Document findings and comparison of prompt versions.

OUTPUT

Prompt Engineering Examples and Outputs:

Example 1: REST API Code Generation

Prompt v1 (Basic): "Write a REST API in Node.js"

Output Quality: Generic, missing error handling, no validation ■■

Prompt v2 (Refined): "Create a production-ready REST API in Node.js with Express for a User resource with CRUD operations. Include input validation using Joi, error handling middleware, and proper HTTP status codes. Use async/await pattern."

Output Quality: Complete, well-structured, production-ready ■■■■■

Key Improvement: Specifying framework, patterns, validation library, and quality requirements produced significantly better output.

Example 2: Cloud Configuration Template

Prompt v1: "Create a Dockerfile"

Output Quality: Basic single-stage build, no optimization ■■

Prompt v2: "Create a multi-stage Dockerfile for a Node.js 20 application that: uses alpine base for smaller image size, separates build and runtime stages, runs as non-root user, includes health check, and copies only production dependencies."

Output Quality: Optimized, secure, production-ready ■■■■■■

Example 3: Business Logic Generation

Prompt: "Generate a discount calculation engine in JavaScript that supports: percentage discounts, flat discounts, buy-X-get-Y-free, coupon code validation with expiry dates, and stacking rules where only the best discount applies. Include unit tests."

Output Quality: Comprehensive logic with edge cases handled ■■■■

Prompt Engineering Best Practices Identified:

1. Be specific about technology stack and version
2. Mention design patterns and coding standards expected
3. Include non-functional requirements (security, performance)
4. Request tests alongside the code
5. Specify output format (modular, commented, typed)
6. Use iterative refinement - evaluate and re-prompt

RESULT

Successfully demonstrated Vibe Coding and Prompt Engineering techniques. Iterative prompt refinement significantly improved code quality from basic (2/5) to production-ready (5/5). Key factors: specificity, context, constraints, and quality requirements in prompts.